



پاسخ‌ها لزوماً یکتا نیستند و پاسخ‌های درست دیگر نیز در نظر گرفته می‌شوند.

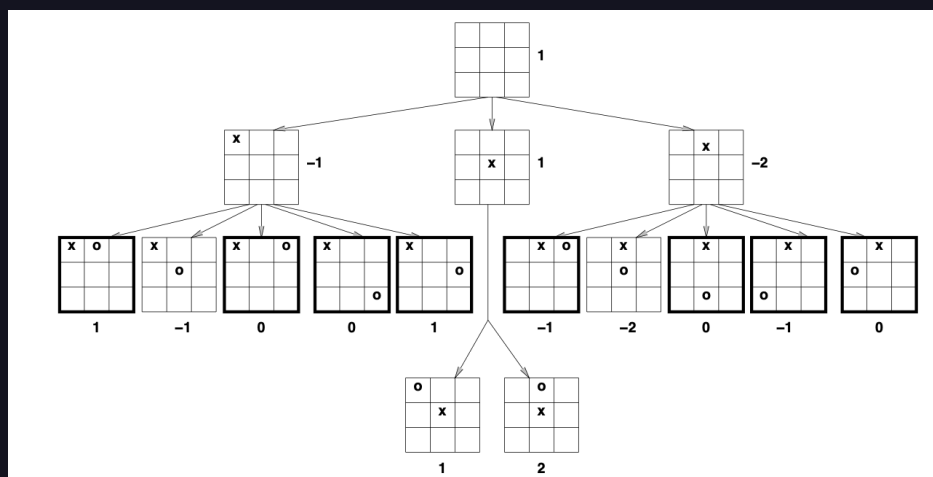
(۱)

گراف حالات، نمایش کلی فضای مسئله است. در این نمایش، هر گره یک حالت ممکن از مسئله را نشان می‌دهد و هر یال نشان‌دهنده‌ی عملی است که باعث انتقال از یک حالت به حالت دیگر می‌شود. در گراف حالات، هر حالت واقعی معمولاً فقط یک بار نمایش داده می‌شود؛ بنابراین ممکن است چند مسیر مختلف به یک گره مشترک برسند و حتی در بعضی مسائل، چرخه نیز وجود داشته باشد.

درخت جست‌وجو، حاصل باز کردن مسیرهای ممکن از حالت اولیه است. در این نمایش، گره‌ها لزوماً حالت‌های یکتا نیستند؛ بلکه هر گره معمولاً یک مسیر خاص از حالت اولیه تا یک حالت را نشان می‌دهد. بنابراین ممکن است یک حالت یکسان، اگر از چند مسیر مختلف قابل دستیابی باشد، چند بار در درخت جست‌وجو ظاهر شود. درخت جست‌وجو برای اجرای الگوریتم‌های جست‌وجو مانند BFS، DFS، A* و غیره به کار می‌رود.

درخت بازی، نوع خاصی از درخت جست‌وجو است که برای مدل‌سازی بازی‌های چندعاملی، به‌ویژه بازی‌های نوبتی، استفاده می‌شود. در این درخت، هر گره نشان‌دهنده‌ی یک وضعیت از بازی است و هر سطح معمولاً مربوط به نوبت یکی از بازیکنان است. در بازی‌های دو نفره‌ی مجموع-صفر، گره‌ها معمولاً به دو نوع MAX و MIN تقسیم می‌شوند؛ بازیکن MAX تلاش می‌کند مقدار نهایی را بیشینه کند و بازیکن MIN تلاش می‌کند همان مقدار را کمینه کند. برگ‌های درخت بازی معمولاً وضعیت‌های پایانی بازی هستند و با تابع سودمندی یا تابع ارزیابی مقداردهی می‌شوند.

تفاوت اصلی این سه مفهوم در هدف و نحوه‌ی نمایش آن‌هاست. گراف حالات، ساختار کلی همه‌ی حالت‌ها و انتقال‌های ممکن را نشان می‌دهد؛ درخت جست‌وجو، مسیرهای قابل بررسی توسط یک الگوریتم جست‌وجو را از حالت اولیه باز می‌کند؛ و درخت بازی، علاوه بر حالت‌ها و اعمال، نوبت بازیکنان و تقابل اهداف آن‌ها را نیز مدل می‌کند. بنابراین می‌توان گفت درخت جست‌وجو معمولاً از روی گراف حالات ساخته می‌شود، و درخت بازی نیز شکل خاصی از درخت جست‌وجو برای مسائل رقابتی و چندعاملی است.



شکل ۱: شکل ۱: درخت سوال دوم

گره‌هایی که نیازی به بررسی آنها نیست، با مرزبندی Bold نشان داده شده‌اند.

فرض می‌کنیم بازی، دو نفره، نوبتی، قطعی، با اطلاعات کامل و مجموع-صفر یا مجموع-ثابت است. در چنین بازی‌ای، الگوریتم minimax مقدار هر وضعیت را از دید بازیکن MAX محاسبه می‌کند. در گره‌های مربوط به MAX، بیشترین مقدار فرزندان انتخاب می‌شود و در گره‌های مربوط به MIN، کمترین مقدار فرزندان انتخاب می‌شود؛ زیرا فرض می‌کنیم رقیب به صورت بهینه بازی می‌کند و همواره حرکتی را انتخاب می‌کند که سود MAX را تا حد امکان کاهش دهد.

بنابراین مقدار minimax در ریشه، در واقع یک مقدار تضمین‌شده برای MAX است. یعنی اگر MAX مطابق راهبرد minimax بازی کند، حتی در برابر بهترین بازی ممکن از سوی MIN نیز حداقل همین مقدار را به دست خواهد آورد. حال اگر MIN در بعضی از گره‌ها به صورت بهینه بازی نکند، به این معناست که به جای انتخاب فرزندی با کمترین مقدار برای MAX، فرزند دیگری را انتخاب کرده است. مقدار چنین فرزندی نمی‌تواند از مقدار کمینه‌ی همان گره کمتر باشد؛ پس نتیجه‌ی نهایی برای MAX حداقل به اندازه‌ی حالتی خواهد بود که MIN به صورت کاملاً بهینه بازی می‌کرد. به بیان دیگر، خطای MIN نمی‌تواند تضمین minimax را برای MAX بدتر کند؛ بلکه فقط ممکن است باعث بهتر شدن نتیجه‌ی MAX شود.

با این حال، اگر MAX بداند که رقیب در بعضی موقعیت‌ها به صورت غیر بهینه عمل می‌کند، ممکن است بتواند با انتخاب حرکتی غیر از حرکت minimax، نتیجه‌ی بهتری در برابر همان رقیب خاص به دست آورد. البته این کار تضمین بدترین حالت را کاهش می‌دهد و فقط زمانی منطقی است که رفتار غیر بهینه‌ی رقیب قابل پیش‌بینی باشد.

برای مثال، فرض کنید MAX در ریشه دو انتخاب A و B دارد. اگر A را انتخاب کند، بازی بلافاصله با مقدار 2 برای MAX تمام می‌شود. اگر B را انتخاب کند، نوبت به MIN می‌رسد و MIN می‌تواند بین دو نتیجه‌ی 0 و 10 برای MAX یکی را انتخاب کند. اگر MIN بهینه بازی کند، مقدار 0 را انتخاب می‌کند؛ بنابراین مقدار minimax شاخه‌ی B برابر 0 است و MAX باید شاخه‌ی امن‌تر A را با مقدار 2 انتخاب کند.

اما اگر MAX بداند که این رقیب خاص در شاخه‌ی B معمولاً انتخاب اشتباه انجام می‌دهد و مقدار 10 را باقی می‌گذارد، آنگاه انتخاب B می‌تواند در برابر این رقیب خاص بهتر از انتخاب minimax باشد. با این حال، این انتخاب دیگر تضمین minimax را ندارد؛ زیرا اگر MIN برخلاف انتظار بهینه بازی کند، نتیجه‌ی MAX به 0 کاهش می‌یابد.

ترتیب بازديد گره‌ها يا move ordering تأثیر بسیار مهمی بر کارایی الگوریتم alpha-beta pruning دارد. منظور از ترتیب مطلوب این است که در گره‌های MAX، فرزندی که مقدار minimax بزرگ‌تری دارند زودتر بررسی شوند، و در گره‌های MIN، فرزندی که مقدار minimax کوچک‌تری دارند زودتر بررسی شوند. به بیان دیگر، در ترتیب کاملاً مطلوب، در هر گره ابتدا همان حرکتی بررسی می‌شود که در نهایت توسط بازیکن صاحب آن گره انتخاب خواهد شد.

درستی الگوریتم alpha-beta pruning به ترتیب بازديد گره‌ها وابسته نیست؛ یعنی مستقل از اینکه گره‌ها با چه ترتیبی بررسی شوند، در نهایت همان مقداری را به دست می‌آورد که الگوریتم minimax معمولی به دست می‌آورد. با این حال، تعداد گره‌هایی که هرس می‌شوند، به شدت به ترتیب بازديد وابسته است. اگر در گره‌های MAX حرکات خوب زودتر و در گره‌های MIN حرکات بدتر برای MAX زودتر بررسی شوند، مقدارهای α و β سریع‌تر محدودکننده می‌شوند و شاخه‌های بیشتری قابل حذف خواهند بود.

در بدترین حالت، اگر ترتیب بازديد گره‌ها نامطلوب باشد، alpha-beta pruning ممکن است تقریباً به اندازه‌ی minimax معمولی گره بررسی کند. اما در حالت ایده‌آل، یعنی زمانی که ترتیب گره‌ها کاملاً مطلوب باشد، تعداد گره‌های بررسی‌شده به طور قابل توجهی کاهش می‌یابد و پیچیدگی زمانی از حدود

$$O(b^d)$$

به حدود

$$O(b^{d/2})$$

می‌رسد. بنابراین، در این حالت می‌توان با همان مقدار زمان، تقریباً تا دو برابر عمق بیشتر را جست‌وجو کرد. با این حال، باید به یک نکته‌ی مهم توجه کرد. اگر واقعاً ترتیب کاملاً مطلوب را از قبل بدانیم، یعنی بدانیم در هر گره کدام حرکت بهترین حرکت است، در واقع بخش مهمی از پاسخ مسئله را از قبل در اختیار داریم. در چنین حالتی، استفاده از alpha-beta pruning به عنوان روشی برای کشف بهترین حرکت چندان معنادار نیست؛ زیرا خود ترتیب مطلوب می‌تواند ما را مستقیماً به سمت بهترین انتخاب هدایت کند. به بیان دیگر، دانستن ترتیب کاملاً مطلوب معمولاً مستلزم دانستن یا تخمین بسیار دقیق مقدار حرکات است.

بنابراین پاسخ دقیق این است که alpha-beta pruning با ترتیب کاملاً مطلوب از نظر نظری بسیار کارآمد است، اما در عمل چنین ترتیبی معمولاً از قبل در دسترس نیست. به همین دلیل، برنامه‌های بازی‌ساز تلاش می‌کنند با استفاده از روش‌های تقریبی، مانند تابع ارزیابی، جست‌وجوی کم‌عمق‌تر، iterative deepening، یا تجربه‌ی قبلی از موقعیت‌های مشابه، ترتیب خوبی برای بررسی حرکات پیدا کنند. پس ترتیب مطلوب، کارایی جست‌وجو را افزایش می‌دهد، اما اگر این ترتیب به صورت کامل و دقیق از قبل معلوم باشد، خود نیاز به جست‌وجو تا حد زیادی زیر سؤال می‌رود.

۱. در بازی‌های دو نفره، شرط zero-sum حالت خاصی از شرط constant-sum است. در بازی zero-sum داریم:

$$U_{MAX}(s) + U_{MIN}(s) = 0$$

اما در بازی constant-sum داریم:

$$U_{MAX}(s) + U_{MIN}(s) = c$$

که در آن c یک مقدار ثابت است.

در نتیجه، در بازی دو نفره constant-sum می‌توان نوشت:

$$U_{MIN}(s) = c - U_{MAX}(s)$$

بنابراین اگر بازیکن MIN بخواهد سود خودش را بیشینه کند، در واقع باید U_{MAX} را کمینه کند؛ زیرا هرچه U_{MAX} بیشتر شود، مقدار U_{MIN} کمتر خواهد شد. پس در بازی‌های دو نفره constant-sum، حتی اگر بازی دقیقاً zero-sum نباشد، منطق الگوریتم کلاسیک minimax همچنان برقرار است.

به بیان دیگر، درستی minimax وابسته به این نیست که مجموع سودها حتماً صفر باشد؛ بلکه کافی است مجموع سود دو بازیکن ثابت باشد. در این صورت، بیشینه‌سازی سود یک بازیکن معادل کمینه‌سازی سود بازیکن دیگر است.

۲. اگر در یک بازی دو نفره نه شرط zero-sum برقرار باشد و نه شرط constant-sum، به طور کلی نمی‌توان از alpha-beta pruning سنتی استفاده کرد. دلیل این است که در چنین بازی‌ای، بیشینه کردن سود بازیکن MIN الزاماً معادل کمینه کردن سود بازیکن MAX نیست.

برای مثال، فرض کنید در یکی از گره‌های مربوط به بازیکن دوم، دو نتیجه‌ی زیر وجود داشته باشد:

$$A = (0, 5), \quad B = (10, 6)$$

که مؤلفه‌ی اول سود MAX و مؤلفه‌ی دوم سود بازیکن دوم است. اگر بازیکن دوم عقلانی و خودخواهانه بازی کند، گزینه‌ی B را انتخاب می‌کند، چون سود خودش در آن 6 است، در حالی که در A سود او 5 است. اما اگر این گره را مانند یک گره MIN کلاسیک تفسیر کنیم، گزینه‌ی A انتخاب می‌شود، چون مقدار MAX در آن کمتر است.

بنابراین، در بازی‌های دو نفره‌ی غیر مجموع-ثابت، فرض اصلی minimax کلاسیک شکسته می‌شود: رقیب لزوماً سود MAX را کمینه نمی‌کند، بلکه سود خودش را بیشینه می‌کند. چون alpha-beta pruning سنتی بر همین ساختار minimax تکیه دارد، استفاده از آن به شکل کلاسیک ممکن است شاخه‌هایی را حذف کند که در واقع می‌توانستند روی تصمیم نهایی اثر بگذارند. البته اگر بازی را با یک مدل متفاوت، مثلاً با فرض رفتار خصمانه‌ی کامل نسبت به MAX، بازتعریف کنیم، می‌توان روی آن مدل خاص جست‌وجو انجام داد؛ اما این دیگر تحلیل استاندارد بازی عقلانی غیر مجموع-ثابت نیست.

۳. در بازی‌های با بیش از دو بازیکن، حتی اگر شرط zero-sum یا constant-sum برقرار باشد، به طور کلی نمی‌توان از alpha-beta pruning به روش سنتی استفاده کرد. دلیل اصلی این است که در بازی دو نفره‌ی مجموع-ثابت، سود بازیکن دوم مستقیماً با سود MAX رابطه‌ی معکوس دارد؛ اما در بازی چندبازیکنه چنین رابطه‌ی ساده‌ای میان سود یک بازیکن و سود MAX وجود ندارد.

برای مثال، در یک بازی سه‌نفره‌ی constant-sum با مجموع ثابت 10، فرض کنید در نوبت بازیکن دوم دو گزینه‌ی زیر وجود داشته باشد:

$$A = (8, 2, 0), \quad B = (1, 1, 8)$$

که مؤلفه‌ها به ترتیب سود بازیکن اول، دوم و سوم هستند. اگر نوبت بازیکن دوم باشد، او گزینه‌ی A را انتخاب می‌کند، چون سود خودش در A برابر 2 و در B برابر 1 است. اما از دید بازیکن اول، گزینه‌ی B بسیار بدتر است، چون سود او را به 1 کاهش می‌دهد. بنابراین بازیکن دوم الزاماً گزینه‌ای را انتخاب نمی‌کند که سود بازیکن اول را کمینه کند.

این مثال نشان می‌دهد که حتی با ثابت بودن مجموع سودها، در بازی چندبازیکنه نمی‌توان گره‌های مربوط به سایر بازیکنان را به سادگی مانند گره‌های MIN نسبت به بازیکن اول در نظر گرفت. هر بازیکن سود خودش را بیشینه می‌کند، نه اینکه الزاماً سود یک بازیکن خاص را کمینه کند. از این رو، alpha-beta pruning سنتی که بر ساختار دو نفره‌ی MIN/MAX تکیه دارد، به طور کلی برای بازی‌های چندبازیکنه معتبر نیست.

۴. در بازی‌های با بیش از دو بازیکن و در نبود هر دو شرط zero-sum و constant-sum، به طور کلی استفاده از alpha-beta pruning سنتی معتبر نیست. در این حالت، نه تنها بیش از یک رقیب وجود دارد، بلکه میان سود بازیکنان نیز رابطه‌ی ثابت و ساده‌ای برقرار نیست. بنابراین نمی‌توان گفت که تصمیم یک بازیکن دیگر الزاماً باعث کمینه شدن سود MAX می‌شود.

در چنین بازی‌هایی معمولاً به جای مقدار عددی واحد، باید یک بردار سود در نظر گرفت؛ برای مثال:

$$(U_1, U_2, \dots, U_n)$$

و در نوبت هر بازیکن i ، آن بازیکن معمولاً مؤلفه‌ی U_i را بیشینه می‌کند. چون تصمیم هر بازیکن بر اساس مؤلفه‌ی خودش انجام می‌شود، نمی‌توان با استفاده از تنها دو کران α و β برای سود MAX، شاخه‌ها را به طور ایمن حذف کرد.

بنابراین پاسخ کلی منفی است: alpha-beta pruning به روش سنتی برای بازی‌های چندبازیکنه‌ی غیر مجموع-ثابت قابل استفاده نیست. برای چنین بازی‌هایی باید از روش‌های دیگری مانند الگوریتم maxn، مدل‌های مبتنی بر بردار سود، یا روش‌های خاص و اصلاح‌شده‌ی هرس استفاده کرد. این روش‌ها ممکن است در برخی شرایط بتوانند شاخه‌هایی را حذف کنند، اما دیگر همان alpha-beta pruning کلاسیکِ بازی‌های دو نفره‌ی مجموع-ثابت نیستند.

۱. هنگامی که به دلیل محدودیت زمانی یا حافظه، جست‌وجو را پیش از رسیدن به حالت‌های پایانی واقعی متوقف می‌کنیم، مقدار وضعیت‌های مرزی با استفاده از یک تابع ارزیابی تخمینی محاسبه می‌شود. در این حالت دو مسئله‌ی مهم به وجود می‌آید: horizon effect و non-quietescent positions.

منظور از non-quietescent position وضعیتی است که از نظر بازی هنوز ناپایدار است و احتمال وقوع تغییرات مهم در چند حرکت بعدی زیاد است. برای مثال، در شطرنج، وضعیتی که در آن یک مهره در معرض گرفته شدن است، یا یک حمله‌ی مستقیم در جریان است، وضعیت آرام و پایدار محسوب نمی‌شود. اگر جست‌وجو دقیقاً در چنین وضعیتی متوقف شود، تابع ارزیابی ممکن است مقدار گمراه‌کننده‌ای تولید کند؛ زیرا اثر واقعی تهدید یا تبادل مهره‌ها هنوز در درخت دیده نشده است.

در مقابل، یک وضعیت quietescent وضعیتی است که در آن انتظار نمی‌رود در چند حرکت بعدی تغییر ناگهانی و مهمی در مقدار ارزیابی رخ دهد. به همین دلیل، در برنامه‌های بازی معمولاً تلاش می‌شود جست‌وجو در وضعیت‌های آرام‌تر متوقف شود، نه در موقعیت‌های ناپایدار. روشی که برای کاهش این مشکل به کار می‌رود، quiescence search نام دارد؛ یعنی اگر وضعیت مرزی هنوز ناپایدار باشد، جست‌وجو برای چند حرکت مهم دیگر ادامه پیدا می‌کند تا به وضعیتی پایدارتر برسیم.

مسئله‌ی دوم، horizon effect است. این مسئله زمانی رخ می‌دهد که یک پیامد مهم، مانند باختن یک مهره یا شکست قطعی، درست بعد از عمق برش قرار داشته باشد و بنابراین توسط الگوریتم دیده نشود. در این حالت، برنامه ممکن است حرکتی را انتخاب کند که فقط آن پیامد بد را چند حرکت به تأخیر می‌اندازد، بدون آنکه واقعاً از آن جلوگیری کند. چون این پیامد پشت «افق» جست‌وجو قرار گرفته است، تابع ارزیابی فعلی نمی‌تواند آن را به درستی تشخیص دهد.

بنابراین، non-quietescent positions به ناپایدار بودن وضعیت‌هایی اشاره دارد که جست‌وجو در آن‌ها متوقف شده است، و horizon effect به خطایی اشاره دارد که به دلیل قرار گرفتن یک پیامد مهم در بیرون از عمق جست‌وجوی فعلی به وجود می‌آید.

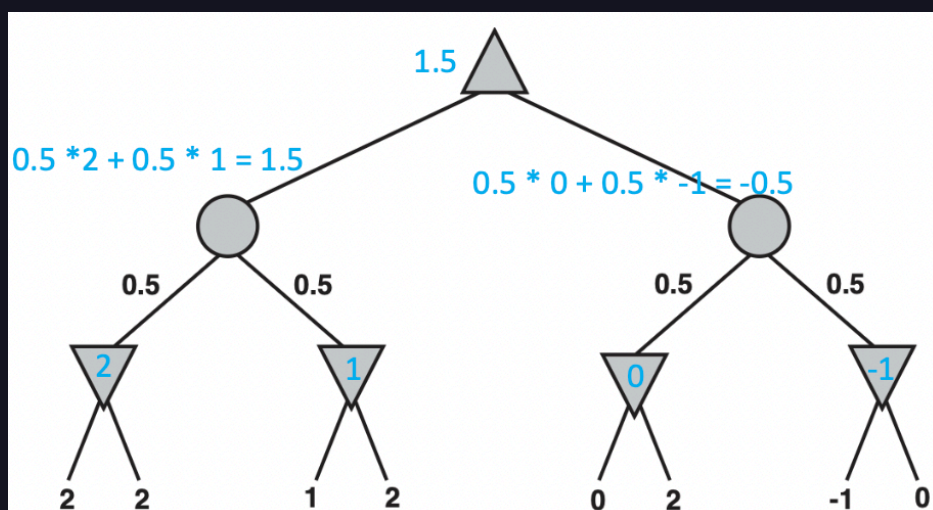
۲. الگوریتم مناسب:

Iterative Deepening Search

در این روش، جست‌وجو ابتدا با عمق کم انجام می‌شود و سپس عمق جست‌وجو به تدریج افزایش می‌یابد. مزیت این روش آن است که اگر زمان جست‌وجو تمام شود، الگوریتم همچنان بهترین نتیجه‌ی کامل‌شده از عمق قبلی را در اختیار دارد. بنابراین، به جای انتخاب یک مقدار ثابت و غیرمنعطف برای عمق برش، می‌توان از جست‌وجوی تدریجی عمیق‌تر استفاده کرد و تصمیم‌گیری را با محدودیت زمانی هماهنگ‌تر ساخت.

همچنین می‌توان با استفاده از نتایج هر مرحله اجرای الگوریتم، تخمینی از ordering move در مراحل بعد بدست بیاوریم که خود اجرای الگوریتم را بهینه‌تر می‌کند و یکی از روش‌های استاندارد برای پیاده‌سازی توابع heuristic به جهت تخمین move ordering بهینه است.

پاسخ سوال هفتم، بخش اول



شکل ۲: شکل ۲: دخت سوال هفتم، بخش اول

پاسخ سوال هفتم، بخش دوم

برای مدل سازی این بازی، باید آن را به صورت یک درخت بازی شامل گره های تصمیم گیری و گره های شانس در نظر بگیریم. از آنجا که دو صفحه ای مستقل از بازی دوز وجود دارد، یک وضعیت بازی را می توان به صورت زیر نمایش داد:

$$s = (B_1, B_2, p)$$

که در آن B_1 و B_2 وضعیت دو صفحه ای دوز هستند و p نشان می دهد نوبت کدام بازیکن است. هر خانه در هر صفحه می تواند یکی از سه مقدار خالی، X ، یا O را داشته باشد. بنابراین، وضعیت بازی باید مشخص کند که در هر یک از دو صفحه، مهره های O و X در کدام خانه ها قرار دارند و بازیکن بعدی چه کسی است.

در این بازی، گره های تصمیم گیری مربوط به بازیکن هستند. اگر نوبت بازیکن X باشد، آن گره را می توان یک گره MAX در نظر گرفت، و اگر نوبت بازیکن O باشد، آن گره را می توان یک گره MIN در نظر گرفت. در یک گره تصمیم گیری، بازیکن باید از میان حرکت های قانونی خود در صفحه ای که توسط سکه مشخص شده است، یک حرکت را انتخاب کند. به عنوان مثال، اگر سکه صفحه ای اول را مشخص کرده باشد، بازیکن فقط مجاز است علامت خود را در یکی از خانه های خالی صفحه ای اول قرار دهد.

از آنجا که در ابتدای هر نوبت یک سکه پرتاب می شود، قبل از گره تصمیم گیری هر بازیکن، یک گره شانس قرار می گیرد. این گره شانس دو فرزند دارد: یکی متناظر با انتخاب صفحه ای اول و دیگری متناظر با انتخاب صفحه ای دوم. اگر سکه عادلانه باشد، احتمال هر یک از این دو حالت برابر است با:

$$P(B_1) = \frac{1}{2}, \quad P(B_2) = \frac{1}{2}$$

پس ساختار کلی هر نوبت به این صورت است:

Chance Node $\rightarrow$ Player Decision Node $\rightarrow$ New Game State

در نتیجه، اگر نوبت بازیکن X باشد، ابتدا یک گره شانس مشخص می‌کند که حرکت باید روی کدام صفحه انجام شود. سپس در گره تصمیم‌گیری، بازیکن X از میان خانه‌های خالی همان صفحه، حرکتی را انتخاب می‌کند که از دید او بهترین باشد. پس از انجام حرکت، وضعیت یکی از دو صفحه تغییر می‌کند و نوبت به بازیکن O می‌رسد. همین روند برای بازیکن O نیز تکرار می‌شود.

حالت‌های پایانی بازی زمانی رخ می‌دهند که یکی از بازیکنان در یکی از دو صفحه، سه علامت خود را در یک سطر، ستون یا قطر کامل کند. می‌توان تابع سودمندی را به صورت زیر تعریف کرد:

$$U(s) = \begin{cases} +1 & \text{در یکی از دو صفحه برنده شود، اگر } X \\ -1 & \text{در یکی از دو صفحه برنده شود، اگر } O \\ -1 & \text{اگر بازی بدون برنده پایان یابد.} \end{cases}$$

برای ارزیابی چنین درختی، الگوریتم مناسب expectiminimax است. در این الگوریتم، مقدار گره‌های تصمیم‌گیری مانند minimax محاسبه می‌شود؛ یعنی گره‌های X مقدار بیشینه و گره‌های O مقدار کمینه‌ی فرزندان خود را انتخاب می‌کنند. اما مقدار گره‌های شانس برابر امید ریاضی مقدار فرزندان است. بنابراین، اگر گره شانس دو فرزند با مقدارهای v_1 و v_2 داشته باشد و سکه عادلانه باشد، مقدار آن برابر است با:

$$\frac{1}{2}v_1 + \frac{1}{2}v_2$$

پس تفاوت اصلی این مدل با درخت بازی معمولی دوز در این است که بازیکن نمی‌تواند آزادانه تصمیم بگیرد روی کدام صفحه حرکت کند. انتخاب صفحه به صورت تصادفی و توسط گره شانس انجام می‌شود، و بازیکن فقط پس از مشخص شدن صفحه می‌تواند حرکت مناسب را روی همان صفحه انتخاب کند.